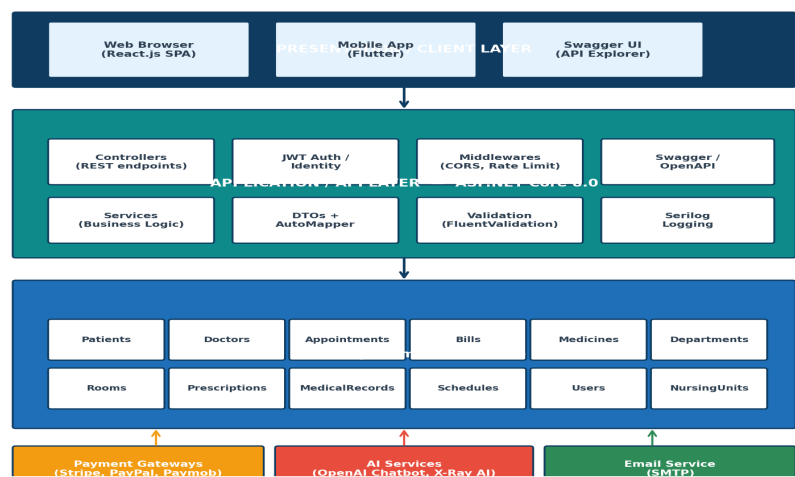


Hospital Management System

Backend Technical Documentation

Figure 1 — Three-Tier Layered Architecture



Project	Al-Hayat Hospital — HMS
Module	Server-Side / RESTful API
Platform	ASP.NET Core 8.0 Entity Framework Core SQL Server 2022
Document Type	Technical Backend Documentation
Version	1.0
Date	June 2026
Audience	Developers, Architects, Reviewers, IT Operations

Table of Contents

1. Abstract	3
2. Introduction	4
3. Project Objectives & Scope	5
4. Technology Stack	6
5. System Architecture	7
6. Request Lifecycle & Middleware Pipeline	9
7. Project Structure & Code Organization	10
8. Data Model & Database Design	11
9. Entity-Relationship Diagram	13
10. Domain Entities Reference	14
11. Component / Module Diagram	16
12. Class Diagram (Domain Model)	17
13. Use-Case Diagram	18
14. Authentication & Authorization	19
15. Authentication Sequence	20
16. Appointment Booking Workflow	21
17. Billing & Payment Flowchart	22
18. Appointment State Diagram	23
19. REST API Reference	24
20. Mock Screens (UI consuming the API)	26
21. Deployment & Hosting	29
22. Security, Logging & Operations	30
23. Conclusion & Future Work	31

1. Abstract

The Hospital Management System (HMS) is a comprehensive, server-side platform designed to digitise and automate the day-to-day administrative and clinical operations of **Al-Hayat Hospital**. It is built on top of **ASP.NET Core 8.0** and exposes a fully documented **RESTful API** consumed by a React web interface and a companion Flutter mobile application. The backend is responsible for the secure handling of all hospital data — including patients, doctors, appointments, electronic medical records, prescriptions, billing and pharmacy inventory.

From an engineering perspective, the system is structured around three logical tiers: a thin **presentation** tier (the front-end clients), an **application** tier that contains the controllers, services and authentication infrastructure, and a **data** tier composed of Entity Framework Core mapped onto a SQL Server 2022 instance. Cross-cutting concerns — including JWT-based authentication, role-based authorization, request validation, structured logging through Serilog and request rate limiting — are all enforced at the API gateway level before any business code is invoked.

This document is intended as the single reference for the backend portion of the platform. It explains the high-level architecture of the API, walks through the request lifecycle, describes the domain model and the database schema, presents UML and flowchart artefacts (use-case, class, sequence, state, activity, component and deployment), enumerates every public REST endpoint and finally provides representative screen mock-ups for the consuming clients. The document concludes with deployment guidelines and a roadmap for future improvements.

Keywords: Hospital Management, ASP.NET Core 8.0, Entity Framework Core, REST API, JWT, SQL Server, Layered Architecture, Microservices-ready, Swagger / OpenAPI.

2. Introduction

2.1 Background

Healthcare institutions operate under continuously increasing volumes of patient data and complex coordination requirements between departments. Paper-based records and disconnected spreadsheets create duplication of effort, slow patient throughput and introduce clinically significant errors. Modern hospitals therefore require an integrated information platform that consolidates patient demographics, appointment scheduling, electronic medical records, pharmacy inventory and billing into a single authoritative data store. Al-Hayat Hospital commissioned the development of such a platform — the Hospital Management System (HMS) — to answer this need.

2.2 Problem Statement

Before the introduction of HMS, daily operations relied on a mixture of manual registers, spreadsheets and isolated departmental applications. Patient histories had to be physically retrieved from archives, appointments were tracked on paper calendars and invoices were generated outside any inventory or service catalogue. The administrative overhead grew proportionally with patient volume and reporting for hospital management was largely reactive.

The HMS backend is the technical answer to these issues. It centralises all data in a relational store, exposes business operations as well-defined REST endpoints, applies strict validation and authentication on every request, and serves both web and mobile front-ends from the same code base.

2.3 Document Purpose

This document focuses exclusively on the **backend** portion of the project. It describes the structural design of the API, the persistence model, the security perimeter and the operational characteristics of the server. The reader is assumed to be familiar with general software engineering concepts and the .NET ecosystem. Front-end implementation details are referenced only where they are needed to understand the contract exposed by the backend.

2.4 Target Audience

- **Backend developers** joining the project who need a high-level orientation before reading the source code.
- **Software architects** evaluating the structural quality of the platform.
- **QA engineers** writing integration and end-to-end test cases against the API.
- **DevOps / IT operations** teams responsible for deploying, monitoring and scaling the system.
- **Academic reviewers** who require a complete artefact describing both the design and the implementation choices of the project.

3. Project Objectives & Scope

3.1 Objectives

The HMS backend has been engineered around six core objectives. Each objective translated directly into one or more architectural decisions that are reflected in the source code:

#	Objective	Architectural Response
O1	Provide a single source of truth for all hospital data.	One relational database, accessed exclusively through HospitalDbContext.
O2	Enforce strong authentication and role-based authorization.	ASP.NET Identity + JWT bearer tokens, with [Authorize(Roles=...)] annotations.
O3	Decouple business rules from data access and presentation.	Service layer (IService / *Service) sits between controllers and DbContext.
O4	Expose every business operation through a RESTful HTTP interface.	RESTful controllers with versioned routes and Swagger/OpenAPI metadata.
O5	Be observable and auditable in production.	Serilog structured logs + daily rolling files + request lifecycle middleware.
O6	Be extensible: new features must not require dependency injections.	Dependency injection, container, interface-driven services, AutoMapper.

3.2 Functional Scope

The backend implements the following functional modules. Each module is represented by a dedicated controller, a set of services and one or more domain entities.

- **Identity & Access:** registration, login, password change, profile lookup, JWT issuance and validation, role assignment (Admin, Doctor, Receptionist, Patient).
- **Patient Management:** CRUD on patient demographics, medical history, allergies, current medications and insurance information.
- **Doctor Management:** doctor profiles, specialisation, license, consultation fees and assignment to departments.
- **Appointment Management:** slot calculation, booking, confirmation, cancellation and completion.
- **Medical Records:** structured diagnoses, treatments, vital signs and lab results.
- **Prescriptions & Pharmacy:** creation of prescriptions, line-item items referencing medicines, dispensing workflow.
- **Billing & Payment:** invoice generation, partial payments, integration with Stripe, PayPal and Paymob payment gateways.
- **Departments, Rooms, Nursing Units:** physical resource catalogue.
- **Dashboard & Reporting:** aggregated KPIs for hospital management.
- **Chatbot & X-Ray AI:** optional AI-powered endpoints that delegate to OpenAI and an external inference service.

3.3 Out of Scope

The backend does not implement the front-end rendering layer, the data-warehouse side of analytical reporting (which would typically use a separate BI tool), or the HL7 / FHIR integration with regional health information exchanges. These items are listed as future work.

4. Technology Stack

The platform is implemented on a mainstream Microsoft stack, augmented with a small number of widely-adopted third-party libraries. The selection emphasises long-term support, strong tooling, security and a clean separation of concerns. The following table groups the technologies by layer.

Layer	Technology	Version	Purpose
Framework	ASP.NET Core	8.0	Web host, routing, DI, middleware pipeline
ORM	Entity Framework Core	8.0	Object-relational mapping & migrations
Database	Microsoft SQL Server	2022	Persistent relational storage
Identity	ASP.NET Identity	8.0	User store, password hashing, role store
Auth	JWT Bearer	8.0	Stateless token-based authentication
Mapping	AutoMapper	12.0	Entity ↔ DTO conversion
Validation	FluentValidation	11.x	Declarative request validation
Logging	Serilog	8.0	Structured logging to console & rolling files
Docs	Swagger / Swashbuckle	6.6	Interactive OpenAPI documentation
Rate-limit	AspNetCore.RateLimiting	8.0	Login throttling & abuse protection
Payments	Stripe / PayPal / Paymob	SDK	External payment gateway integration
AI	OpenAI / Custom X-Ray AI	HTTP	Optional chat and image-diagnosis endpoints
Email	SMTP (SmtpClient)	BCL	Transactional emails (registration, reset)
Container	Docker / docker-compose	—	Reproducible local & staging deployments

4.1 Rationale for ASP.NET Core 8.0

ASP.NET Core 8.0 is the latest long-term-support release of the framework at the time of writing. It offers a mature dependency-injection container, first-class support for minimal-API or controller-based programming models, performant Kestrel hosting and a rich middleware pipeline. Critically, it integrates natively with ASP.NET Identity (for the user store) and EF Core (for data access) so the entire stack can be built with a coherent set of libraries rather than gluing together heterogeneous components.

4.2 Rationale for SQL Server + EF Core

The hospital domain is intrinsically relational. Patients have many appointments, appointments belong to exactly one doctor and one room, prescriptions link patients to medical records and so on. A relational database — modelled here with EF Core code-first migrations on top of SQL Server — naturally expresses these constraints and provides strong guarantees through foreign keys, indexes and ACID transactions.

5. System Architecture

The HMS backend follows a classic **three-tier layered architecture**. Requests originate from a presentation layer (the browser or the Flutter app), traverse a well-defined middleware pipeline, are handled by a controller, are delegated to a service for business processing and finally reach a SQL Server database through Entity Framework Core. The diagram below summarises the layout.

Figure 1 — Three-Tier Layered Architecture

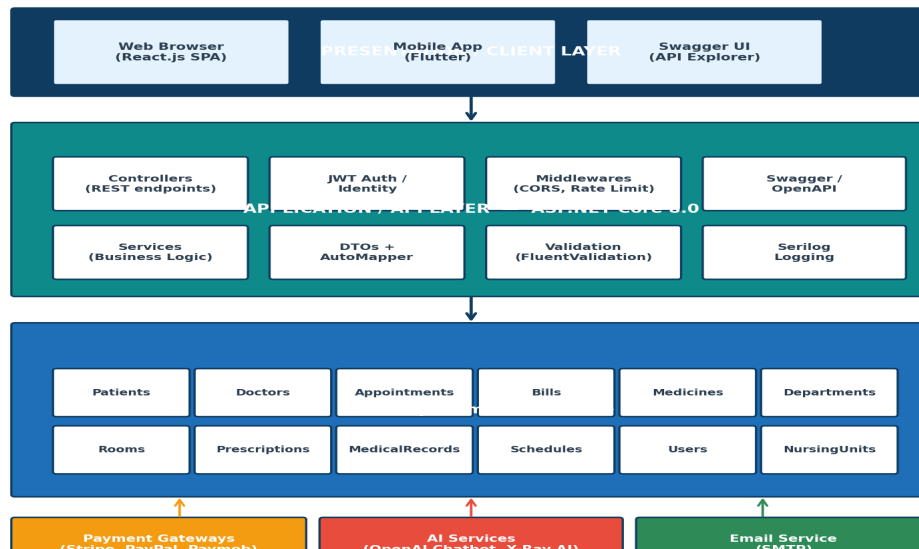


Figure 1 — Three-tier layered architecture of the HMS backend.

5.1 Layers in Detail

5.1.1 Presentation / Client Layer

The client layer is intentionally thin. It consists of a React-based Single-Page Application (SPA) for desktop use, a Flutter cross-platform application for mobile use, and the Swagger UI that ships with the backend for API exploration. All three speak the same language — HTTPS with JSON payloads — and all three rely on the same JWT-based authentication scheme.

5.1.2 Application / API Layer

This is where the bulk of the backend code lives. It is composed of:

- **Controllers** — thin classes that translate HTTP into method calls. Each controller is annotated with `[ApiController]` and `[Route("api/[controller]")]`.
- **Services** — interface + implementation pairs (e.g. `IPatientService / PatientService`) that encapsulate business rules.
- **DTOs** — Data Transfer Objects that flatten and shape the data exposed to clients. They prevent over-posting and decouple the public contract from internal entities.
- **Mappings** — AutoMapper profiles that convert `Entity` ↔ `DTO` in both directions.
- **Middlewares** — JWT authentication, CORS, rate limiting, error handling and Serilog request logging.

5.1.3 Data Layer

The data layer is composed of a single Entity Framework Core `DbContext` — `HospitalDbContext` — that exposes one `DbSet<T>` per domain entity. All relationships, indexes, decimal precisions and seed data are configured fluently inside `OnModelCreating`. SQL Server 2022 is the target database engine.

5.2 Cross-Cutting Concerns

Several concerns affect every request and are therefore handled outside the controllers and services:

- **Authentication** — Identity + JWT, validated by *UseAuthentication()*.
- **Authorization** — Role-based policies declared on controller methods.
- **Validation** — Model state validation, augmented with *FluentValidation* for the more complex DTOs.
- **Logging** — Serilog with a daily rolling file sink under *logs/*.
- **Error handling** — A global *UseExceptionHandler* that returns RFC-7807 *ProblemDetails* payloads.
- **Rate limiting** — A token bucket on *POST /api/auth/login* to mitigate credential-stuffing.

5.3 Component / Module Diagram

Figure 10 below explodes the same architecture into the concrete components that live inside the backend project. Controllers are mapped 1-to-1 to services, and services delegate persistence to a single *HospitalDbContext*. AutoMapper sits on the side and is consumed by both controllers and services to convert between domain entities and the DTOs returned to clients.

Figure 10 — Component / Module Diagram

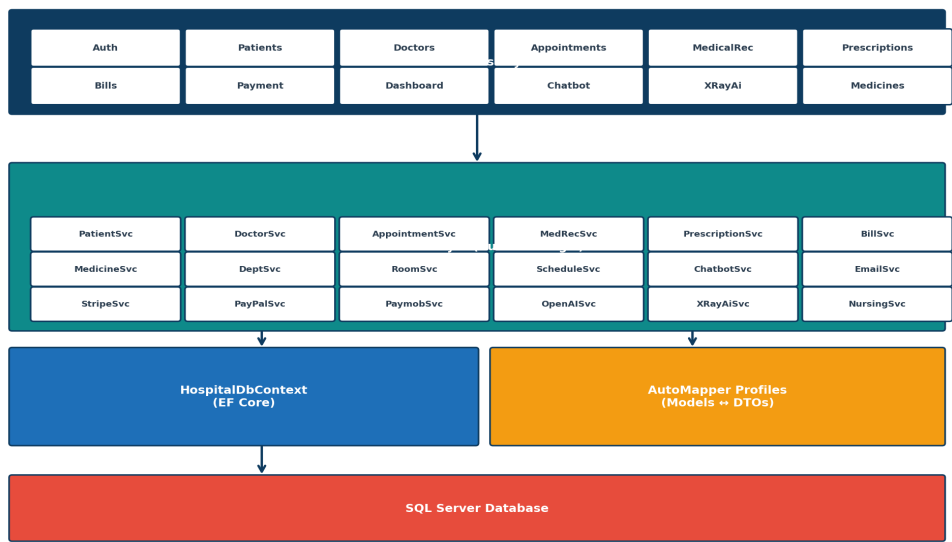


Figure 10 — Component diagram showing controller, service, mapping and data layers.

5.4 Why a Layered Architecture

Although the project could have been written as a single monolithic controller per feature, splitting it into *Controller* → *Service* → *DbContext* was a deliberate choice. The benefits become tangible as soon as more than one developer touches the code base:

- **Testability** — services depend on interfaces, so they can be unit-tested with mocked dependencies without spinning up SQL Server.
- **Reusability** — multiple controllers (REST, GraphQL, gRPC if needed) can share the same business logic.
- **Maintainability** — when business rules change, only the service layer needs to be updated. Controllers stay focused on HTTP plumbing.
- **Security** — DTOs prevent over-posting attacks where a malicious client tries to set internal fields such as *IsActive* or *UserId*.

6. Request Lifecycle & Middleware Pipeline

Every HTTP call that arrives at the HMS backend traverses the same well-defined pipeline before its response is sent back. Understanding this pipeline is essential for debugging, performance tuning and security analysis.

Figure 2 – HTTP Request Lifecycle Through the Backend

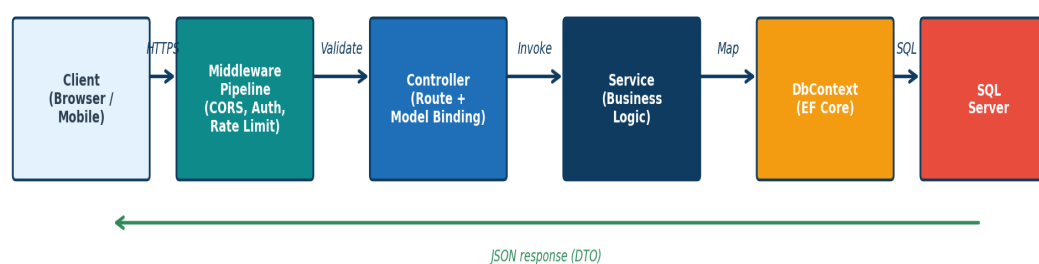


Figure 2 — Request lifecycle from the client to SQL Server.

6.1 Pipeline Stages

Stage	Component	Responsibility
1	HTTPS termination (Kestrel/IIS)	TLS handshake and certificate validation.
2	Serilog request logging	Captures method, path, status and duration of each request.
3	CORS policy	Approves or rejects cross-origin requests based on configured origins.
4	Authentication	Reads the <i>Authorization: Bearer</i> header and validates the JWT.
5	Authorization	Enforces role and policy attributes declared on controller methods.
6	Rate limiting	Applies the <i>auth-login</i> partition for the login endpoint.
7	Routing & model binding	Selects the correct controller action and binds the body to a DTO.
8	FluentValidation	Runs declarative rules on the bound DTO; rejects with 400 if invalid.
9	Controller action	Translates the call into a service-layer invocation.
10	Service layer	Executes business rules; orchestrates several repository calls.
11	EF Core / SQL	Translates LINQ to SQL, executes against SQL Server, materialises entities.
12	AutoMapper	Projects the entity graph into a DTO for the response.
13	Global error handler	Catches unhandled exceptions and returns a ProblemDetails payload.

6.2 Example: Program.cs Excerpt

The pipeline is configured during the application start-up inside *Program.cs*. The simplified excerpt below highlights how the cross-cutting concerns described above are stitched together:

```
var builder = WebApplication.CreateBuilder(args);

builder.Host.UseSerilog();
builder.Services.AddDbContext<HospitalDbContext>(o =>
```

```
o.UseSqlServer(builder.Configuration.GetConnectionString("DefaultConnection")));

builder.Services.AddIdentity<User, IdentityRole>(...)
    .AddEntityFrameworkStores<HospitalDbContext>();

builder.Services.AddAuthentication(JwtBearerDefaults.AuthenticationScheme)
    .AddJwtBearer(opt => opt.TokenValidationParameters = ...);

builder.Services.AddAutoMapper(typeof(Program));
builder.Services.AddScoped<IPatientService, PatientService>();
builder.Services.AddScoped<IDoctorService, DoctorService>();
// ... all other services

var app = builder.Build();
app.UseSerilogRequestLogging();
app.UseCors("AllowAll");
app.UseAuthentication();
app.UseAuthorization();
app.UseRateLimiter();
app.MapControllers();
app.Run();
```

7. Project Structure & Code Organization

The backend lives in a single ASP.NET Core project named *Hospital Mangement System*. The folder structure is feature-by-layer: code is grouped first by responsibility (controllers, services, models ...) and then by feature. This is the most familiar layout for .NET developers and makes it easy to locate the code corresponding to any given endpoint.

Figure 13 — Project Folder Structure

Hospital Management System/
Program.cs
appsettings.json
Hospital Mangement System.csproj
Controllers/ (22 REST controllers)
Services/ (interface + implementation)
Models/ (17 domain entities)
DTOs/ (14 transfer objects)
Data/ (HospitalDbContext + Seed)
Mappings/ (AutoMapperProfile.cs)
Migrations/ (EF Core migrations)
Configuration/(JWT, ConnectionString resolver)
Properties/ (launchSettings.json)
logs/ (Serilog daily rolling logs)

Figure 13 — Top-level folder layout of the backend project.

7.1 Key Folders

- **Controllers/** — 22 REST controllers, one per resource (Auth, Patients, Doctors, Appointments, MedicalRecords, Prescriptions, Bills, Payment, Medicines, Departments, Rooms, Schedules, Dashboard, NursingUnits, Chatbot, XRayAi, Health, Features, Seed, ...).
- **Services/** — Business logic. Each service is declared as an interface (*I*Service.cs*) and an implementation (**Service.cs*).
- **Models/** — Domain entities. They inherit from a common *BaseEntity* class that provides *Id*, *CreatedAt*, *UpdatedAt*, *IsActive*.
- **DTOs/** — Lightweight contracts used to communicate with clients. Every controller action returns a DTO, never an entity directly.
- **Data/** — *HospitalDbContext*, *SeedData* and helpers.
- **Mappings/** — A single *AutoMapperProfile.cs* that registers all the Entity ↔ DTO mappings.
- **Migrations/** — EF Core code-first migrations.
- **Configuration/** — Connection-string resolver, seed-password provider.
- **Properties/** — *launchSettings.json* for local development.
- **logs/** — Daily rolling logs produced by Serilog (e.g. *hospital-management-20260612.txt*).

7.2 Source-Code Statistics

Metric	Value
Number of Controllers	22
Number of Services (interface + impl pairs)	18 pairs
Number of Domain Entities	17
Number of DTOs	14

Lines of C# Code (approx.)	≈ 12 000
Database Tables (including Identity)	20+
REST Endpoints	80+

8. Data Model & Database Design

The persistence layer is built on Entity Framework Core code-first migrations. All domain entities derive from a common *BaseEntity* class that ensures every table has an *Id*, *CreatedAt*, *UpdatedAt* and *IsActive* column. ASP.NET Identity contributes the *AspNetUsers*, *AspNetRoles* and related tables out of the box.

8.1 BaseEntity Definition

```
public abstract class BaseEntity
{
    public int Id { get; set; }
    public DateTime CreatedAt { get; set; } = DateTime.UtcNow;
    public DateTime? UpdatedAt { get; set; }
    public bool IsActive { get; set; } = true;
}
```

8.2 Domain Entities

The full domain consists of 17 entities. The table below summarises the most important ones and the role they play in the system:

Entity	Role
User	ASP.NET Identity user — the authentication principal.
Patient	Person receiving care; demographics, history, insurance.
Doctor	Clinician with specialisation, license and consultation fee.
Staff	Non-clinical personnel (administrators, receptionists).
Department	Hospital department (Cardiology, Pediatrics, ...).
Room	Physical room attached to a department; has type & capacity.
NursingUnit	Sub-unit inside a department for nursing assignments.
Appointment	Scheduled slot between patient, doctor and room.
MedicalRecord	Diagnosis & treatment captured during/after an appointment.
Prescription	Header object grouping a list of <i>PrescriptionItem</i> .
PrescriptionItem	Single line of a prescription (qty, dosage, frequency).
Medicine	Pharmacy item with stock, expiry, unit price.
Bill	Patient invoice composed of multiple <i>BillItem</i> .
BillItem	Line item billed to the patient (service, drug or room fee).
Schedule	Doctor weekly availability pattern.
Feature	Feature-flag table for opt-in capabilities.

8.3 Database Schema Overview

Figure 14 below presents a colour-coded overview of the tables that make up the HMS database. Each colour groups tables that belong to the same logical sub-domain (identity, patient/doctor master data, scheduling, clinical records, billing & pharmacy, administration).

Figure 14 — Database Schema Overview

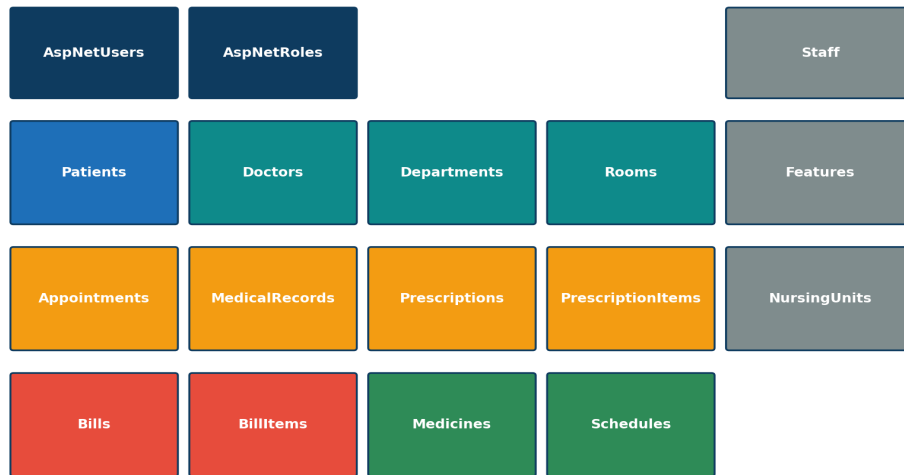


Figure 14 — Database schema overview by logical sub-domain.

8.4 Important Design Decisions

- **Soft-delete** via *IsActive* instead of physical removal. This preserves audit trails and prevents orphaned foreign keys.
- **DeleteBehavior.Restrict** on most relationships. Cascading deletes are explicitly avoided to keep medical history intact.
- **Indexes** on every business key (e.g. *NationalId* on Patient, *LicenseNumber* on Doctor) and on every foreign key, configured fluently in *HospitalDbContext.OnModelCreating*.
- **Decimal precision** explicitly set on monetary columns (*HasColumnType("decimal(18,2)")*) to avoid implicit conversions.
- **Seed data** for the admin user, default roles, departments and rooms is loaded the first time the application starts.

8.5 Migrations

Schema changes are managed exclusively through EF Core migrations. The day-to-day workflow is:

```
# 1) Add a migration after editing a model
dotnet ef migrations add <DescriptiveName>

# 2) Apply it to the local database
dotnet ef database update

# 3) Roll back if needed
dotnet ef database update <PreviousMigrationName>
```

9. Entity-Relationship Diagram

The Entity-Relationship Diagram below depicts the persistent model of the HMS backend. It shows every primary entity, the foreign keys that bind them and the cardinality of each relationship.

Identity-related tables (*AspNetUsers*, *AspNetRoles*, ...) are summarised under the single *User* box for readability.

Figure 3 — Entity-Relationship Diagram (Core Entities)

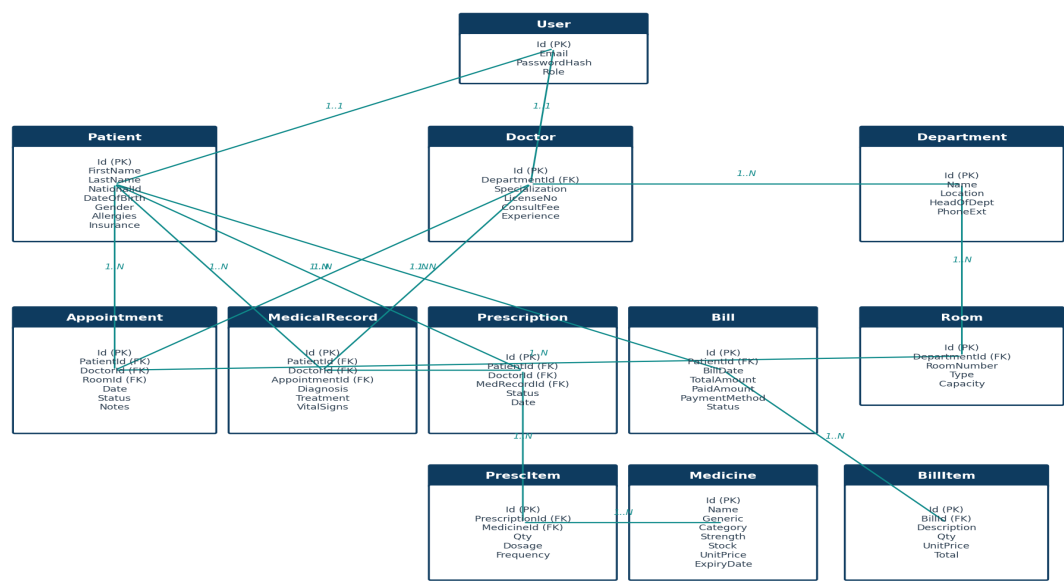


Figure 3 — Entity-Relationship Diagram of the HMS database.

9.1 Reading the Diagram

- Each box represents a single database table. The dark band at the top is the table name; the body lists primary and foreign keys plus the most relevant attributes.
- Lines between boxes represent foreign-key relationships. Annotations such as *1..N* indicate cardinality.
- All foreign keys are *ON DELETE RESTRICT*; clinical history cannot be accidentally lost by deleting a parent record.

10. Domain Entities Reference

This section documents the most important columns of each entity in tabular form. Only business-relevant columns are listed; *BaseEntity* columns (*Id*, *CreatedAt*, *UpdatedAt*, *IsActive*) are implicit on every table.

10.1 Patient

Column	Type	Notes
FirstName, LastName	nvarchar(100)	Required.
NationalId	nvarchar(20)	Unique index.
Email	nvarchar(256)	Required, indexed.
PhoneNumber, PhoneNumber2	nvarchar(20)	Primary + optional secondary.
DateOfBirth	datetime2	Used for age computation.
Gender	nvarchar(10)	Male / Female / Other.
Address	nvarchar(500)	Free text.
EmergencyContactName/Phone	nvarchar	Nullable.
InsuranceProvider/Number	nvarchar	Nullable; supports unbilled patients.
MedicalHistory, Allergies, CurrentMedications	nvarchar(500)	Clinical context.
UserId	nvarchar	FK to AspNetUsers.

10.2 Doctor

Column	Type	Notes
FirstName, LastName	nvarchar(100)	Required.
LicenseNumber	nvarchar(50)	Unique.
Specialization	nvarchar(100)	e.g. Cardiology.
Qualifications	nvarchar(500)	Education / certifications.
ExperienceYears	int	Years of practice.
ConsultationFee	decimal(18,2)	Default fee per session.
DepartmentId	int	FK to Departments.

10.3 Appointment

Column	Type	Notes
PatientId, DoctorId, RoomId	int	Foreign keys.
AppointmentDate	datetime2	Calendar date of the appointment.

StartTime, EndTime	time	Slot boundaries.
Status	nvarchar(20)	Pending, Confirmed, InProgress, Completed, Cancelled, NoShow.
Notes	nvarchar(500)	Optional free text.

10.4 Bill

Column	Type	Notes
PatientId	int	FK to Patients.
BillDate	datetime2	Issuing date.
DueDate	datetime2	Payment deadline.
TotalAmount	decimal(18,2)	Sum of BillItems.
PaidAmount	decimal(18,2)	Amount actually collected.
Status	nvarchar(20)	Unpaid / PartiallyPaid / Paid / Overdue.
PaymentMethod	nvarchar(20)	Cash / Card / Stripe / PayPal / Paymob.

10.5 BillItem

Column	Type	Notes
BillId	int	FK to Bills.
Description	nvarchar(200)	e.g. 'Cardiology Consultation'.
Quantity	int	Number of units.
UnitPrice	decimal(18,2)	Per-unit price.
TotalPrice	decimal(18,2)	Quantity × UnitPrice.

10.6 Medicine

Column	Type	Notes
Name, GenericName	nvarchar(100)	Display + chemical name.
Manufacturer	nvarchar(100)	Producer / brand.
Category	nvarchar(50)	Antibiotic, Analgesic, ...
DosageForm, Strength	nvarchar(50)	e.g. 'Tablet 500 mg'.
UnitPrice	decimal(18,2)	Retail price.
StockQuantity	int	Current inventory.
ReorderLevel	int	Threshold for reorder alerts.
ExpiryDate	datetime2	Used by expiry-tracking jobs.

10.7 Prescription / PrescriptionItem

Column	Type	Notes
PatientId, DoctorId, MedicalRecordId	int	Foreign keys.
PrescriptionDate	datetime2	Date written.
Status	nvarchar(20)	Pending, Dispensed, Cancelled.
Items: MedicineId	int	FK to Medicines.
Items: Quantity	int	Units prescribed.
Items: Dosage	nvarchar(50)	e.g. '20 mg'.
Items: Frequency	nvarchar(50)	e.g. 'twice daily'.
Items: Duration	nvarchar(50)	e.g. '7 days'.

11. Class Diagram (Domain Model)

The class diagram below presents an object-oriented view of the domain. It shows the most important domain classes, their attributes and the behaviour they expose. All entities inherit from *BaseEntity*, which is the only abstract class in the model. Methods such as *Confirm()*, *Cancel()*, *Complete()* on *Appointment* are mapped to state transitions enforced by *AppointmentService*.

Figure 11 — Simplified Class Diagram (Domain Model)

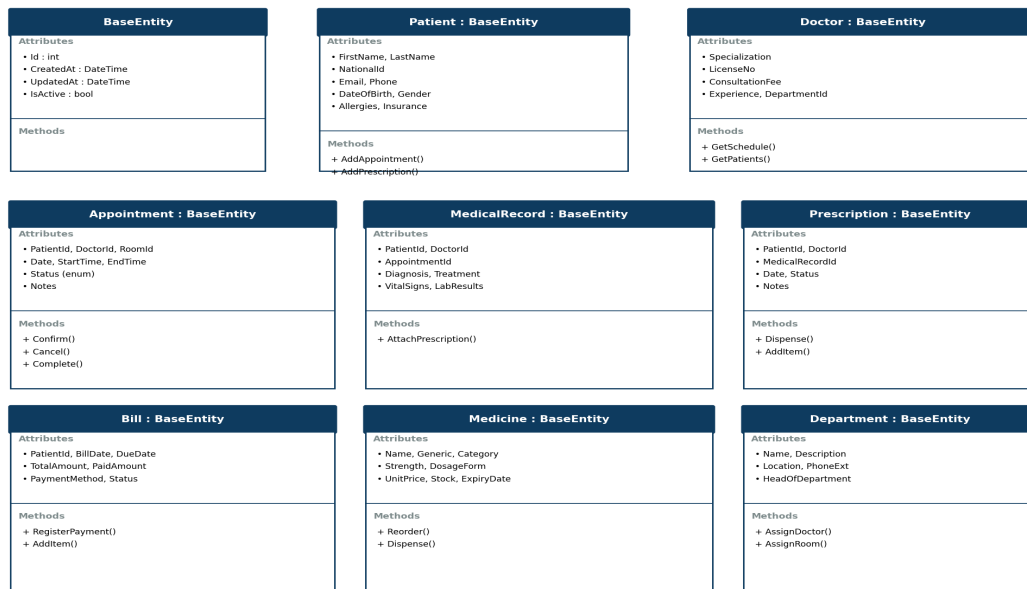


Figure 11 — Simplified class diagram of the HMS domain model.

11.1 Inheritance

Inheritance is used very sparingly. *BaseEntity* exists only to factor out auditing columns. All other relationships are expressed through composition and navigation properties rather than class hierarchies. This is intentional: deep inheritance in a persistence model tends to interact poorly with EF Core's table-per-hierarchy and table-per-type strategies.

11.2 Behaviour vs. Anaemia

Entities in this project carry primarily data, while orchestration logic lives in the corresponding services. This is sometimes referred to as an *anaemic domain model*. The trade-off is deliberate: services are easier to unit-test and to share between several controllers, and they keep transactional boundaries explicit. When a behaviour is intrinsic to a single entity (e.g. recomputing the total of a bill after an item is added), it is implemented as a method on the entity.

12. Use-Case Diagram

The use-case diagram summarises the actors that interact with the system and the high-level capabilities exposed to each of them. The same physical user may actually own multiple actor roles (for example, a doctor who also holds a receptionist account for the front desk) — the diagram refers to roles, not individuals.

Figure 6 — Use-Case Diagram (Actors & System Boundary)

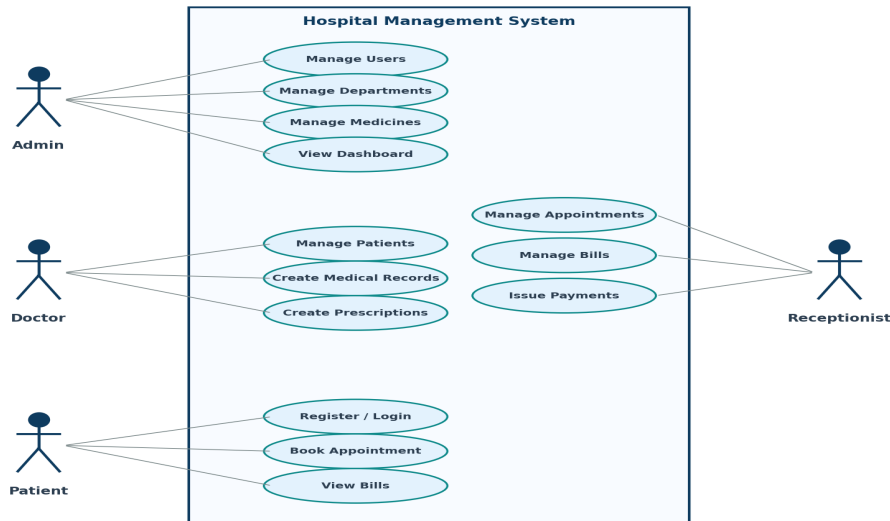


Figure 6 — Actors and high-level use cases of the HMS.

12.1 Actors

- **Admin** — Full access. Creates other users, configures departments, manages the pharmacy catalogue and reviews global KPIs from the dashboard.
- **Doctor** — Sees their own patients, fills in medical records, prescribes medicines and reads appointment schedules.
- **Receptionist** — Creates appointments and bills on behalf of patients, registers payments at the front desk.
- **Patient** — Self-service: registers, logs in, books appointments and views personal medical history and bills.

12.2 Authorization Mapping

Each use case is implemented by one or more REST endpoints and protected with the appropriate `[Authorize(Roles=...)]` attribute. The mapping is enforced by ASP.NET Identity, which inspects the role claims contained in the JWT issued at login.

13. Authentication & Authorization

13.1 Strategy Overview

Authentication in HMS is stateless and based on **JSON Web Tokens (JWT)**. Credentials are presented exactly once, at the `POST /api/auth/login` endpoint, in exchange for a signed token. The token is then attached to every subsequent request through the `Authorization: Bearer <token>` header. No server-side session state is kept, which makes the API trivially horizontally scalable.

13.2 Identity Configuration

User and role storage is delegated to ASP.NET Identity, which writes to the same SQL Server database used by the rest of the application. The password policy configured for HMS is:

```
options.Password.RequireDigit          = true;
options.Password.RequiredLength        = 8;
options.Password.RequireUppercase      = true;
options.Password.RequireLowercase      = true;
options.Password.RequireNonAlphanumeric = false;
options.User.RequireUniqueEmail        = true;
```

13.3 JWT Configuration

JWT signing uses HS256 with a secret key sourced from configuration. Issuer, audience, lifetime and signing key are all validated on every request:

```
options.TokenValidationParameters = new TokenValidationParameters
{
    ValidateIssuer          = true,
    ValidateAudience        = true,
    ValidateLifetime         = true,
    ValidateIssuerSigningKey = true,
    ValidIssuer              = jwt["Issuer"],
    ValidAudience           = jwt["Audience"],
    IssuerSigningKey         = new SymmetricSecurityKey(Encoding.UTF8.GetBytes(secret))
};
```

13.4 Roles

Four roles are defined and seeded the first time the application starts: **Admin**, **Doctor**, **Receptionist** and **Patient**. They are stored in the *AspNetRoles* table and assigned to users through *AspNetUserRoles*. Each role is asserted on the controller side via *[Authorize(Roles="...")]*.

14. Authentication Sequence

The sequence diagram below details the interactions that take place between the client, the AuthController, the ASP.NET Identity *UserManager*, the JWT service helper and the underlying database when a user attempts to log in. The scenario depicted is the *happy path* where the credentials match an existing active user.

Figure 4 — JWT Authentication Sequence

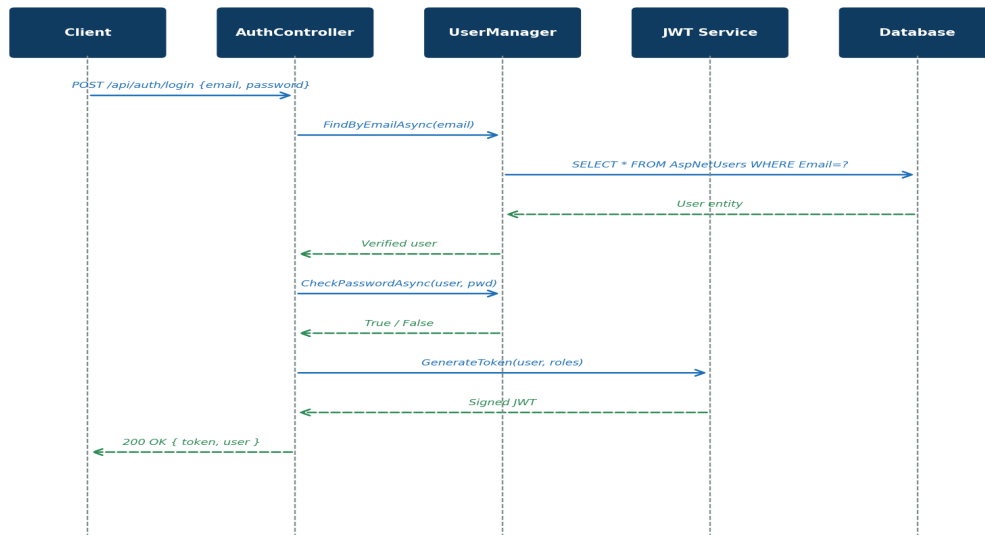


Figure 4 — JWT login sequence.

14.1 Failure Paths

- **Email not found.** *FindByEmailAsync* returns *null*; the controller responds with HTTP 401 *Unauthorized* and a generic message — the exact reason is intentionally not leaked.
- **Wrong password.** *CheckPasswordAsync* returns *false*; same 401 response.
- **Rate limit exceeded.** The *auth-login* token bucket rejects the request with HTTP 429 *Too Many Requests* before the controller is invoked.
- **Locked account.** ASP.NET Identity automatically locks the account after repeated failed attempts (configurable in *appsettings.json*).

14.2 Token Lifetime & Renewal

Tokens are short-lived (60 minutes by default) to mitigate the impact of a leaked token. Refresh tokens are not used; instead the front-end forces a fresh login when the token expires. This trade-off favours simplicity over UX continuity and is appropriate for a hospital intranet.

14.3 Registration Activity

Figure 12 below shows the alternative flow for new-user registration. It follows the same validation-then-persistence pattern but performs additional role assignment afterwards.

Figure 12 — User Registration Activity Diagram

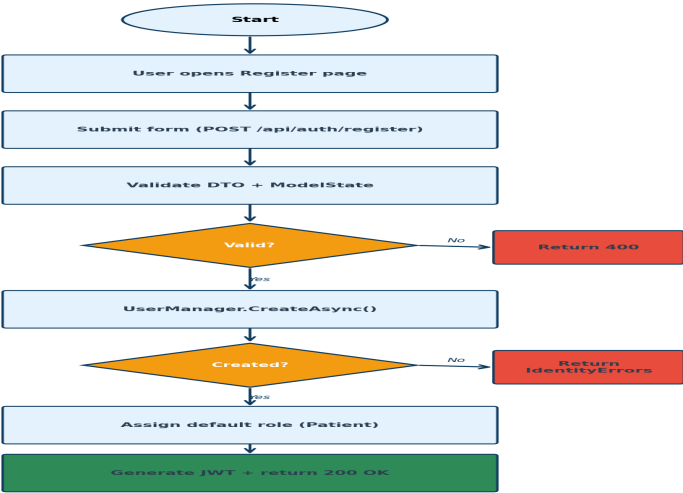


Figure 12 — User registration activity diagram.

15. Appointment Booking Workflow

Booking an appointment is the most frequent write operation against the HMS backend, and also the one with the most complex validation. The system must ensure that the requested doctor is available, that the slot does not collide with an existing appointment and that the assigned room is free at the same time. All of this is encapsulated inside `AppointmentService.CreateAsync`, which is invoked by the controller after model-state validation.

Figure 5 — Appointment Booking Sequence

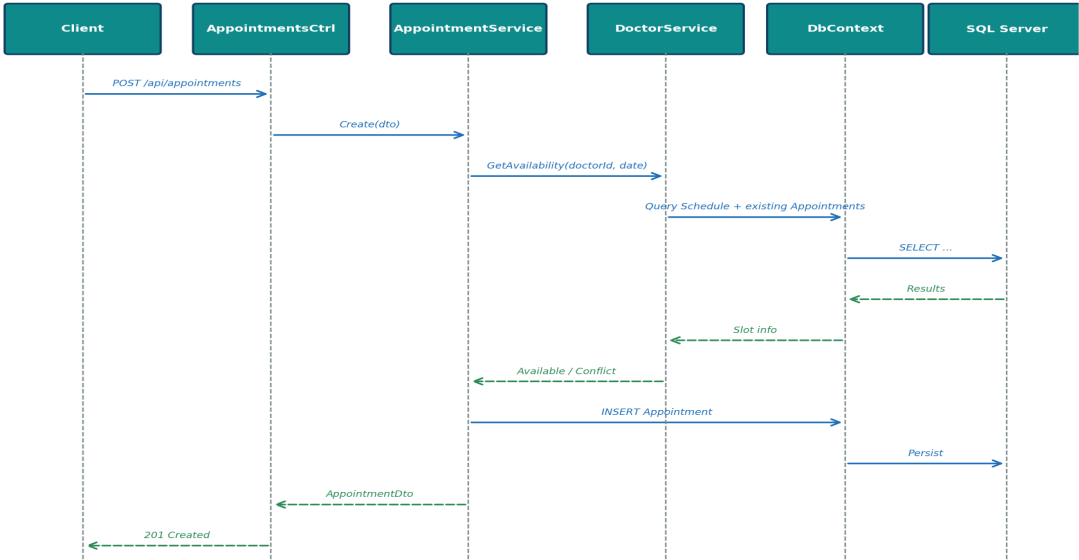


Figure 5 — End-to-end sequence for booking an appointment.

15.1 Slot Computation

Availability is computed by intersecting the doctor's weekly *Schedule* (defined in the *Schedules* table) with the set of appointments already registered for the same day. The first 30-minute opening is offered to the client as a candidate; the client may then pick a later slot from the returned list.

15.2 Concurrency

To avoid double-booking when two clients try to grab the same slot simultaneously, the service performs the availability check and the insertion inside a single database transaction. A SQL Server unique filtered index on (*DoctorId*, *AppointmentDate*, *StartTime*) acts as the last line of defence: even if a race condition slips through the service layer, the database will reject the duplicate.

15.3 Cancellation and Completion

Two dedicated endpoints — *PUT* `/api/appointments/{id}/cancel` and *PUT* `/api/appointments/{id}/complete` — are exposed for status transitions. Both invoke *AppointmentService* methods that validate the current state before transitioning, in accordance with the state diagram in Section 17.

16. Billing & Payment Flowchart

Billing is the second-most complex workflow in the platform. A bill is built incrementally as services are delivered to a patient — consultations, lab tests, drugs dispensed by the pharmacy and room charges all generate line items. The controller exposes three operations: *create*, *add item* and *register payment*. The flowchart below tracks a bill from creation through completion.

Figure 8 — Bill Creation & Payment Flowchart

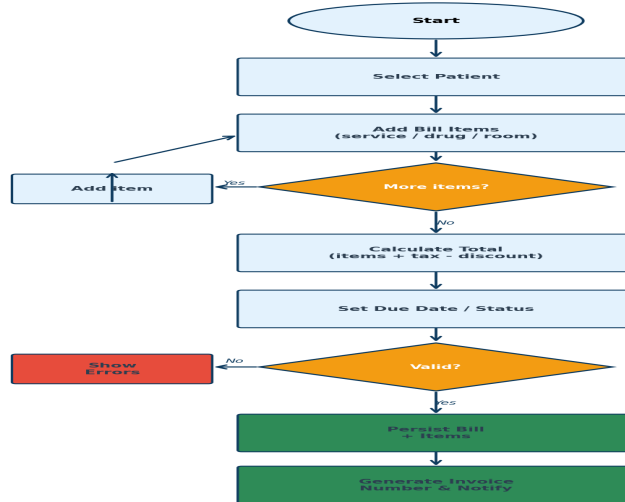


Figure 8 — Bill creation and payment flowchart.

16.1 Payment Gateways

Three external payment gateways are pre-integrated: **Stripe** (default), **PayPal** and **Paymob** (for local Egyptian payments). They sit behind a common *IPaymentGatewayService* interface; switching providers is a single registration line in *Program.cs*:

```
builder.Services.AddScoped<IPaymentGatewayService, StripePaymentService>();
builder.Services.AddScoped<PayPalPaymentService>();
builder.Services.AddHttpClient<PaymobPaymentService>();
```

16.2 Insurance

When a patient has an active insurance entry (*InsuranceProvider* + *InsuranceNumber*), the bill is flagged accordingly. The actual reimbursement process is delegated to the hospital's accounting back-office and is out of scope for the digital workflow described here — but the data is captured to make it straightforward in the future.

17. Appointment State Diagram

Appointments traverse a constrained set of states. The state machine below documents every legal transition. Any attempt to move from a state to a non-adjacent state is rejected by *AppointmentService* with a *400 Bad Request* response.

Figure 7 — Appointment Status State Diagram

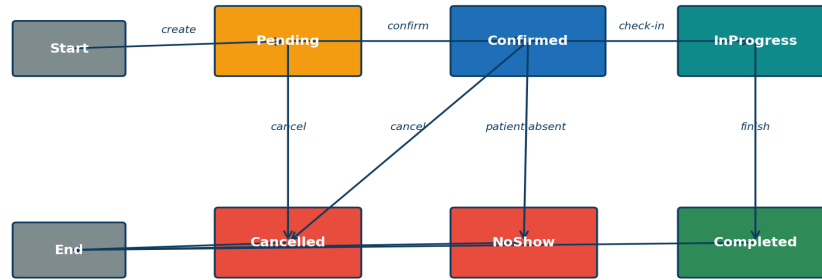


Figure 7 — Appointment state diagram.

17.1 Transitions Explained

- **Pending → Confirmed.** The receptionist or the doctor approves the booking.
- **Pending / Confirmed → Cancelled.** Triggered either by the patient (within the allowed window) or by the staff.
- **Confirmed → InProgress.** The patient is checked in at the front desk.
- **Confirmed → NoShow.** A scheduled background job promotes appointments that passed their start time without a check-in.
- **InProgress → Completed.** The doctor finalises the appointment, typically after writing a Medical Record and (optionally) a Prescription.

17.2 Side Effects

State transitions are not just record updates; they also trigger downstream side-effects that the service layer orchestrates:

- Transitioning to **Cancelled** frees the room and the doctor's slot so they become available for re-booking.
- Transitioning to **Completed** can automatically generate a draft Bill containing the consultation fee.
- Transitioning to **NoShow** may flag the patient for follow-up by the front desk.

18. REST API Reference

The backend exposes more than 80 REST endpoints across 22 controllers. Below is a curated summary of the most important ones, grouped by resource. The full, machine-readable specification is available at `/swagger/v1/swagger.json` at runtime.

18.1 Authentication

Method	Route	Auth	Purpose
POST	/api/auth/register	Public	Register a new patient account.
POST	/api/auth/login	Public	Authenticate and receive a JWT.
POST	/api/auth/change-password	User	Update the current user's password.
GET	/api/auth/profile	User	Return the authenticated user profile.

18.2 Patients

Method	Route	Auth	Purpose
GET	/api/patients	Staff	Paged list with optional search filter.
GET	/api/patients/{id}	Staff	Retrieve a patient by id.
POST	/api/patients	Staff	Create a new patient record.
PUT	/api/patients/{id}	Staff	Update an existing patient record.
DELETE	/api/patients/{id}	Admin	Soft-delete a patient.

18.3 Doctors & Departments

Method	Route	Auth	Purpose
GET	/api/doctors	User	List active doctors with department & rating.
POST	/api/doctors	Admin	Onboard a new doctor.
GET	/api/doctors/{id}/schedule	User	Retrieve doctor's weekly availability.
GET	/api/departments	User	List departments.
POST	/api/departments	Admin	Create a department.
GET	/api/rooms	Staff	List rooms (optionally filter by dept).

18.4 Appointments

Method	Route	Auth	Purpose
GET	/api/appointments	Staff	Paged list with status & date filters.
GET	/api/appointments/{id}	Staff	Retrieve a single appointment.
POST	/api/appointments	User	Book a new appointment.

PUT	/api/appointments/{id}	Staff	Reschedule / update an appointment.
PUT	/api/appointments/{id}/cancel	User	Cancel an existing appointment.
PUT	/api/appointments/{id}/complete	Doctor	Mark as completed after the visit.

18.5 Medical Records & Prescriptions

Method	Route	Auth	Purpose
GET	/api/medicalrecords	Staff	Paged list of medical records.
GET	/api/medicalrecords/{id}	Staff	Retrieve a single medical record.
POST	/api/medicalrecords	Doctor	Create a record from an appointment.
PUT	/api/medicalrecords/{id}	Doctor	Update diagnosis / treatment.
GET	/api/prescriptions	Staff	List prescriptions (per patient/doctor).
POST	/api/prescriptions	Doctor	Issue a new prescription.
PUT	/api/prescriptions/{id}/dispense	Staff	Dispense an active prescription.

18.6 Billing & Payment

Method	Route	Auth	Purpose
GET	/api/bills	Staff	Paged list with status filter.
GET	/api/bills/{id}	Staff	Retrieve a single bill (with items).
POST	/api/bills	Staff	Create a new bill.
POST	/api/bills/{id}/payment	Staff	Register a (partial) payment.
POST	/api/payment/checkout/{billId}	Patient	Open a hosted Stripe/PayPal checkout.
POST	/api/payment/webhook	Public	External gateway notification endpoint.

18.7 Pharmacy & Inventory

Method	Route	Auth	Purpose
GET	/api/medicines	Staff	Paged list with low-stock filter.
GET	/api/medicines/{id}	Staff	Retrieve a single medicine.
POST	/api/medicines	Admin	Add a new medicine to the catalogue.
PUT	/api/medicines/{id}	Admin	Update a medicine.
DELETE	/api/medicines/{id}	Admin	Soft-delete a medicine.

18.8 Dashboard, Chatbot & X-Ray AI

Method	Route	Auth	Purpose
GET	/api/dashboard/stats	Staff	Aggregated KPIs.
GET	/api/dashboard/recent-appointments	Staff	Last N appointments.
GET	/api/dashboard/revenue	Admin	Revenue breakdown by service.
POST	/api/chatbot/ask	User	Ask the medical chatbot.

POST	/api/xrayai/analyze	Doctor	Upload an X-Ray for AI analysis.
GET	/api/health	Public	Liveness probe.

18.9 Common Response Format

All non-error endpoints return JSON envelopes shaped as the corresponding DTO. Errors follow the RFC 7807 Problem-Details specification. A typical 400 looks like:

```
{
  "type": "https://tools.ietf.org/html/rfc7231#section-6.5.1",
  "title": "One or more validation errors occurred.",
  "status": 400,
  "errors": {
    "Email": [ "The Email field is required." ]
  },
  "instance": "/api/auth/register"
}
```

19. Mock Screens (UI consuming the API)

Although this document focuses on the backend, it is useful to illustrate the look-and-feel of the clients that consume the REST API. This section gathers representative screen mock-ups for the most frequently used flows. Every screen corresponds to one or more API endpoints described in Section 18.

19.1 Swagger / OpenAPI Explorer

The Swagger UI ships with the backend and is mounted at `/swagger`. It is the fastest way for developers to discover endpoints, inspect request and response schemas and try the API live with a valid JWT.

Screen 1 — Swagger / OpenAPI Explorer

https://localhost:7102/swagger/index.html		
Hospital Management System API v1.0 /swagger/v1/swagger.json		
POST	/api/auth/login	User login (returns JWT)
POST	/api/auth/register	Register a new patient account
GET	/api/patients	List all patients (paged)
POST	/api/patients	Create new patient
GET	/api/patients/{id}	Get patient by id
PUT	/api/patients/{id}	Update patient
DELETE	/api/patients/{id}	Soft-delete patient
GET	/api/doctors	List doctors
POST	/api/appointments	Book appointment
PUT	/api/appointments/{id}/cancel	Cancel appointment
GET	/api/dashboard/stats	Dashboard counters & KPIs
POST	/api/bills	Create bill

Screen 1 — Swagger UI listing of the public endpoints.

19.2 Login Screen

The login screen submits credentials to `POST /api/auth/login`. Successful authentication stores the returned JWT in local storage on web and in secure storage on mobile, then redirects the user to the dashboard.

Screen 2 — Authentication Screen (Login)

https://hms.alhayat.com/login

AI-Hayat Hospital

Hospital Management System

Sign in to your account

Email

admin@hospital.com

Password

.....

Role

Admin ▾

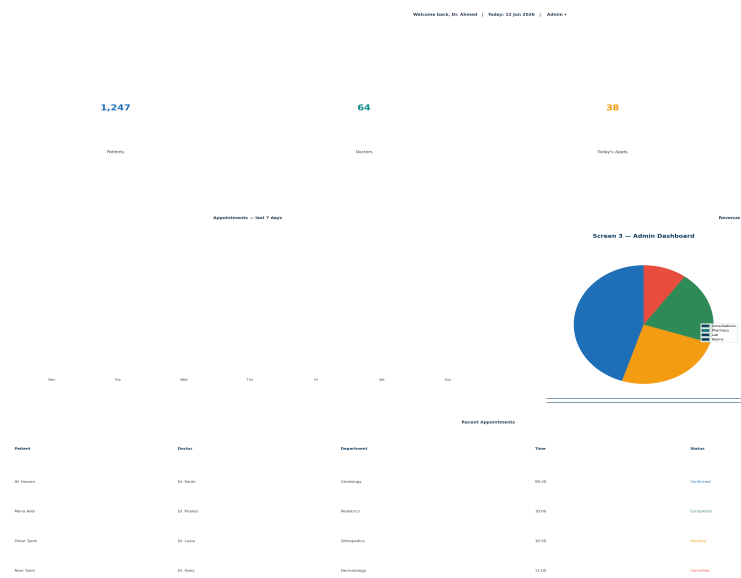
LOG IN

[Forgot password?](#) | [Create account](#)

Screen 2 — Login screen (web client).

19.3 Admin Dashboard

The dashboard aggregates the metrics returned by `GET /api/dashboard/stats`, `/recent-appointments` and `/revenue`. Cards show counts and revenue, the bar chart visualises appointments over the last seven days, and the pie chart splits revenue by service type.



Screen 3 — Hospital administrator dashboard.

19.4 Patients List

The patients list demonstrates the standard CRUD UI offered by the platform. Server-side paging is performed by `GET /api/patients?page=...&size=...` and search filters are forwarded as query-string parameters.

Screen 4 — Patients Management

Dashboard Patients Doctors Appointments Medical Records Prescriptions Bills	Patients / Manage						
	Search by name, national id...					Filter ▾	+ New Patient
	ID	Name	National ID	Gender	Phone	Insurance	Actions
	P-1001	Ali Hassan	29812345678901	Male	010-1234567	MediPlus	🔍✖
	P-1002	Mona Adel	29603456789012	Female	011-2345678	Bupa	🔍✖
	P-1003	Omar Tarek	29005678910123	Male	012-3456789	—	🔍✖
	P-1004	Nour Sami	29701234567890	Female	010-9876543	MetLife	🔍✖
	P-1005	Khaled Mansour	28912345678901	Male	011-4567890	Allianz	🔍✖
	P-1006	Salma Reda	29207654321098	Female	012-5556677	GIG	🔍✖
	P-1007	Hany Adel	28503216549870	Male	010-1112233	AXA	🔍✖
P-1008	Yara Mahmoud	29805671230456	Female	011-3334455	MediPlus	🔍✖	
P-1009	Tarek Ezzat	27704561237890	Male	012-7778899	—	🔍✖	
P-1010	Reem Salah	29509832145678	Female	010-2223344	Bupa	🔍✖	
< 1 2 3 4 >							

Screen 4 — Patients management screen with search & paging.

19.5 Book Appointment

When the user picks a department and a doctor, the front-end calls `GET /api/doctors/{id}/schedule?date=...` to compute and display the available slots. Selecting a slot and submitting the form triggers a `POST /api/appointments`.

Screen 5 — Book Appointment Form

Appointments / New

Schedule New Appointment

Patient

Ali Hassan (P-1001) ▼

Department

Cardiology ▼

Doctor

Dr. Sarah Mansour ▼

Appointment Date

2026-06-18

Available Slots

09:00 - 09:30 | 10:00 - 10:30 ✓ | 11:00 - 11:30

Room

Room 204 ▼

Notes

Patient reports chest tightness on exertion.

Cancel

Save

Screen 5 — Appointment booking form.

19.6 Billing & Invoice

The billing screen consolidates a patient's services and items into a single invoice document. The *Pay Now* button initiates a Stripe / PayPal / Paymob checkout depending on the patient's selected method; the result is communicated back to the backend through the `/api/payment/webhook` endpoint.

Screen 6 — Billing & Payments

Dashboard

Patients

Doctors

Appointments

Bills

Bills / Invoice #INV-2026-0421

Patient

Ali Hassan (P-1001)

Insurance: MediPlus

Doctor: Dr. Sarah Mansour

Date: 2026-06-12 09:30

Status

Partially Paid

Due: 2026-07-12

Method: Stripe

Description	Qty	Unit Price	Total
Cardiology Consultation	1	\$ 80	\$ 80
ECG Test	1	\$ 45	\$ 45
Echocardiogram	1	\$ 120	\$ 120
Atorvastatin 20mg (30 tabs)	1	\$ 22	\$ 22
Aspirin 100mg (60 tabs)	1	\$ 8	\$ 8
		Subtotal	\$ 275
		Total	\$ 275

Pay Now

Print

Screen 6 — Detailed invoice with line items and payment actions.

19.7 Client / Server Interaction

Every screen above ultimately submits an HTTP request to the backend through a thin axios (web) or Dio (mobile) HTTP client. A small interceptor attaches the JWT to outgoing requests, retries idempotent calls on transient failures and redirects to the login screen when the server returns 401. None of this is visible to the rest of the UI codebase, which always interacts with the API through a single, strongly-typed client object.

20. Deployment & Hosting

The backend is designed to run anywhere ASP.NET Core 8.0 is supported: Windows Server, Linux containers or a managed cloud service such as Azure App Service. The reference topology used by the hospital is illustrated in Figure 9 below.

Figure 9 — Deployment Diagram

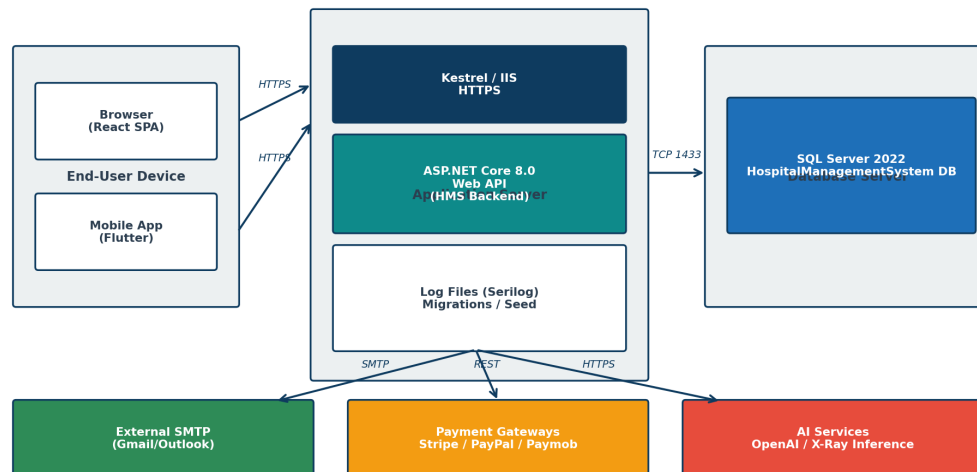


Figure 9 — Reference deployment topology.

20.1 Application Server

The application server hosts ASP.NET Core through Kestrel, behind either IIS (on Windows) or an Nginx reverse proxy (on Linux). HTTPS termination is performed by the reverse proxy. The server also writes daily rolling log files into *logs/*, which can be shipped to a centralised log aggregator (Seq, ELK, Azure Application Insights, ...).

20.2 Database Server

SQL Server 2022 is the default database engine. Daily backups, transaction-log shipping and a regular integrity check (DBCC CHECKDB) are configured at the instance level. EF Core migrations are applied during the deployment phase (*dotnet ef database update*).

20.3 External Services

- **SMTP** — Used by the *EmailService* to deliver transactional emails. Credentials are stored in user secrets / environment variables, never in source.
- **Payment gateways** — Stripe, PayPal and Paymob credentials are configured per environment.
- **AI services** — OpenAI for the chatbot, custom inference for X-Ray. Both are optional and disabled by default.

20.4 Containerised Deployment

The project ships with a *Dockerfile* and a *docker-compose.yml* that build a multi-stage image (SDK → publish → runtime) and pair it with a SQL Server container for local testing. The same image is suitable for production:

```
docker compose build
docker compose up -d
# Backend reachable at https://localhost:7102/swagger
```

21. Security, Logging & Operations

21.1 Defence-in-Depth

Security inside HMS is approached as a series of overlapping layers. Even if one layer is bypassed, the others stop the attack before sensitive data leaks. The main layers are:

- **Transport.** TLS 1.2+ is mandatory; HTTPS is enforced by *UseHttpsRedirection()*.
- **Authentication.** JWT bearer tokens, signed with a symmetric key stored outside source control.
- **Authorization.** Role-based via *[Authorize(Roles=...)]*; sensitive endpoints also use policy-based authorization where ownership of the resource is required.
- **Input validation.** Both data-annotation attributes and FluentValidation rules are run on every DTO. DTOs intentionally omit sensitive fields like *UserId* to prevent over-posting.
- **Output encoding.** All responses are JSON; HTML injection at the API layer is structurally impossible.
- **SQL injection.** Prevented by EF Core parameterised queries.
- **Rate limiting.** The login endpoint is protected by a token-bucket policy.
- **CORS.** Only configured front-end origins are allowed to send credentials.
- **Secrets.** Connection strings and JWT secrets are pulled from environment variables or user-secrets; they are never committed to source control.

21.2 Logging Strategy

Serilog is configured to write to both the console (helpful in development) and to a daily rolling file under *logs/*. Each log entry includes the HTTP verb, the path, the response status, the duration in milliseconds and, when available, the user id taken from the JWT. The same enriched events are exported to an external sink in production for long-term retention and dashboards.

21.3 Health Checks

A lightweight *GET /api/health* endpoint returns a 200 status code with the current build hash. The endpoint is intentionally unprotected so that load balancers and orchestrators can use it without storing credentials.

21.4 Backups & Disaster Recovery

The database is backed up nightly (full) and every fifteen minutes (transaction-log). Backups are encrypted and copied to off-site storage. The recovery point objective (RPO) is 15 minutes and the recovery time objective (RTO) is one hour. The application server is treated as cattle: a fresh deployment from CI can restore service in under ten minutes.

22. Conclusion & Future Work

22.1 Summary

The Hospital Management System backend offers a complete, secure and extensible platform for the digital management of Al-Hayat Hospital. Built on ASP.NET Core 8.0 and Entity Framework Core, the platform brings together identity management, appointment scheduling, electronic medical records, pharmacy inventory and a fully featured billing engine into a single coherent API. The layered architecture — controllers on top of services on top of EF Core — makes the codebase easy to test, easy to extend and easy to onboard new engineers into.

The choices documented in the previous sections (JWT authentication, RESTful endpoints, AutoMapper-based DTOs, Serilog logging, Docker packaging) reflect a deliberate preference for industry-standard, low-friction tooling. They keep the system approachable to any .NET developer while leaving plenty of room for future growth.

22.2 Future Work

A non-exhaustive backlog of improvements has emerged during the project and is tracked for upcoming iterations:

- **Refresh tokens.** Add a refresh-token endpoint so JWTs can be renewed transparently without forcing a new login.
- **Caching.** Introduce Redis to memoise expensive reads on the dashboard and the doctors / departments endpoints.
- **CQRS for reporting.** Split reads and writes for the dashboard to scale the two workloads independently and offer richer analytics.
- **HL7 / FHIR.** Expose a FHIR endpoint for interoperability with regional health information exchanges and other hospital information systems.
- **Real-time notifications.** Add SignalR hubs so the front-end can react instantly to appointment changes and bill updates.
- **Multi-tenancy.** Generalise the platform so several hospitals can share the same code base while keeping their data fully isolated.
- **Observability.** Wire up OpenTelemetry traces and metrics; export them to Prometheus + Grafana or Azure Application Insights.
- **Mobile push notifications.** Send appointment reminders to the Flutter app via Firebase Cloud Messaging.

22.3 Closing Remarks

From a strategic point of view, the platform is now ready to support day-to-day operations of a mid-sized hospital and to serve as a foundation for the more ambitious data-driven initiatives outlined above. Continuous monitoring, incremental improvements and a strong test culture will keep it healthy and maintainable for many years to come.

End of Document

© 2026 Al-Hayat Hospital — Hospital Management System — Backend Technical Documentation v1.0